

SQL

视图

视图：不是逻辑模型的一部分，但作为虚拟关系对用户可见

视图定义

CREATE VIEW：创建视图，基于 SQL 语句的结果集的可视化的表，视图包含行和列，就像一个真实的表。

```
create view v as <query expression>;  
-- <query expression> 可以是任何合法的查询表达式，v表示视图名
```

```
CREATE VIEW HighRatedMovies AS  
SELECT title, rating  
FROM Movies  
WHERE rating >= 8.0;
```

SQL查询中使用视图

在查询中，视图名可以出现在关系名可以出现的任何地方。

```
SELECT COUNT(*)  
FROM HighRatedMovies  
WHERE rating >= 9.0;
```

物化视图

物化视图：如果用于定义视图的实际关系改变，视图也跟着修改

维护：保持物化视图一直在最新状态的过程称为物化视图维护 或 视图维护

视图更新

一个视图是可更新的，如果其满足以下条件：

- from子句中只有一个数据库关系
- select子句只包含关系的属性名，不包含表达式、聚集或distinct声明
- 任何没有出现在select子句中的属性都没有not null约束，同时也不是主码的一部分
- 查询中不含有group by或having子句
- 下面这个例子符合要求

```
CREATE VIEW YoungEmployees AS
SELECT ID, Name, Age, DeptID
FROM Employees
WHERE Age < 30;
```

- 对该视图的更新操作（会直接转化为对基表的操作）：

```
-- 1. 插入数据
-- 注意：插入的数据必须满足视图定义中的 WHERE 条件，
-- 或者是基表中允许为空或有默认值的列。
INSERT INTO YoungEmployees (ID, Name, Age, DeptID)
VALUES (101, 'Alice', 25, 1);

-- 2. 更新数据
UPDATE YoungEmployees
SET Age = 26
WHERE ID = 101;

-- 3. 删除数据
DELETE FROM YoungEmployees
WHERE ID = 101;
```

事务

事务由查询和更新语句的序列组成

事务结束标志

- Commit：提交当前事务
- Rollback：回滚当前事务

```
TRANSACTION;
  SQL语句1;
  SQL语句2;
  ...
  SQL语句n;
COMMIT;
```

索引

索引：创建在关系的属性上，允许数据库高效地找到关系中那些在属性上取给定值的元组，而不用扫描关系中的所有元组

示例代码

```
-- 1. 创建简单索引
-- 在 student 表的 ID 列上创建索引
CREATE INDEX studentID_index ON student(ID);

-- 2. 创建唯一索引
-- 保证 name 列的值唯一，同时提高查询效率
CREATE UNIQUE INDEX name_index ON student(name);

-- 3. 创建复合索引（多列索引）
-- 当查询条件中经常同时包含 dept 和 age 时使用
CREATE INDEX dept_age_index ON student(dept, age);

-- 4. 删除索引
-- 如果不再需要索引，可以删除
DROP INDEX student.studentID_index;
```

授权

对数据的授权包括：

- 授权读取数据
- 授权插入新数据
- 授权更新数据
- 授权删除数据

每种类型的授权都称为一个权限，SQL包括select, insert, update, delete权限，grant语句用来授予权限，revoke语句用来收回权限。

```
GRANT <权限列表>
ON <关系名或视图名>
TO <用户列表>
```

```
REVOKE <权限列表>
ON <关系名或视图名>
FROM <用户列表>
```

- 权限转移：with grant option
- 级联收回：cascade 默认
- 不级联收回：restrict

示例代码

```
-- 1. 授予权限
-- 授予用户 Alice 对 Orders 表的 SELECT 和 INSERT 权限
GRANT SELECT, INSERT ON Orders TO Alice;

-- 2. 授予所有权限
```

```

-- 授予用户 Bob 对 Products 表的所有权限
GRANT ALL PRIVILEGES ON Products TO Bob;

-- 3. 授予权限并允许传递 (WITH GRANT OPTION)
-- 授予 Charlie 对 Employee 表的 SELECT 权限，并允许他将此权限授给其他人
GRANT SELECT ON Employee TO Charlie WITH GRANT OPTION;

-- 4. 收回权限
-- 收回 Alice 对 Orders 表的 INSERT 权限
REVOKE INSERT ON Orders FROM Alice;

-- 5. 级联收回 (CASCADE)
-- 如果 Charlie 已经将 SELECT 权限授给了其他人，
-- 收回 Charlie 的权限时，如果不加 CASCADE 可能会报错（取决于DBMS实现），推荐显示指定
REVOKE SELECT ON Employee FROM Charlie CASCADE;

-- 6. 拒绝权限 (DENY - SQL Server特有)
-- 显式拒绝 David 对 Salary 表的 SELECT 权限，即使他所属的角色有该权限也无效
DENY SELECT ON Salary TO David;

```

触发器

触发器是一条语句，当对数据库做修改时，它自动被系统执行

设置触发器机制的要求：

- 指明什么条件下执行触发器
- 一个引起触发器被检测的事件
- 一个触发器执行必须满足的条件
- 指明触发器执行的动作

示例代码

```

-- SQL Server (SSMS) 触发器语法主要分为 AFTER 和 INSTEAD OF

-- 1. 创建 AFTER 触发器（也可写为 FOR）
-- 场景：当向 Employees 表插入数据后，自动记录日志到 AuditTable
CREATE TRIGGER trg_AfterInsert_Employee
ON Employees
AFTER INSERT
AS
BEGIN
    -- 访问伪表 inserted 获取新插入的数据
    INSERT INTO AuditTable (EmpID, ActionDate, Note)
    SELECT ID, GETDATE(), 'New employee added'
    FROM inserted;
END;

-- 2. 创建 INSTEAD OF 触发器（替代原始操作）
-- 场景：防止直接删除 HighValueCustomers 表中的数据，改为“软删除”（设置状态为 Inactive）

```

```
CREATE TRIGGER trg_InsteadOfDelete_Customer
ON HighValueCustomers
INSTEAD OF DELETE
AS
BEGIN
    -- 不执行真正的 DELETE，而是执行 UPDATE
    UPDATE HighValueCustomers
    SET IsActive = 0
    WHERE CustomerID IN (SELECT CustomerID FROM deleted);

    PRINT 'Data was softly deleted (marked as Inactive).';
END;

-- 3. 禁用与启用触发器
DISABLE TRIGGER trg_AfterInsert_Employee ON Employees;
ENABLE TRIGGER trg_AfterInsert_Employee ON Employees;
```